AZIMI INNOVATION LAB — SESSION 01

The Boundary Between Proposal, Approval, and Execution

SESSION DATE: 20 OCTOBER 2026

Audience, Scope, and Position of the Session

Audience: This session is designed to teach software engineers the architecture and controlled implementation of agentic systems. The learning path rests on a real problem in an existing (Brownfield) system, step-by-step explanation of concepts, inspectable examples, and hands-on exercises. The concepts of Coding Agent, Multi-Agent, Guardrails, and access control are taught in connection with the same scenario.

Engineering problem: In an existing (Brownfield) system, the output of a Coding Agent or specialist agent is only a "change proposal" — not a permit to apply a Patch, run a Migration, change Production settings, or access another Tenant's data. The point where this session connects to Multi-Agent architecture and multilayer Guardrails is the explicit contract between Proposal, Approval, and Execution.

Position of the examples: Excerpts referenced from repositories belonging to Amin Azimi are used as project code samples and as provenance evidence; the independent teaching code of this session is an Educational Adaptation and makes no claim of commercial deployment or any guarantee of Production security.

Suggested continuation of the training (not an approved program): architecture and separation of agent responsibilities → design of Specialized Coding Agents and configuration of Capability/Tool Policy → integration of the controls into Brownfield code and security/regression tests.

Contents — 24 Sections

A linked table of contents of the 24 sections with page numbers; the numbers are computed from the typesetting of this very edition.

01	The Boundary Between Proposal, Approval, and Execution	3
02	90-Minute Teaching Map	3
03	Prerequisites and Success Criteria	4
04	Three-Stage Model with a Fixed Example	4
05	Two Outcome Families That Must Not Be Mixed	5
06	Architecture and Trust Boundary	5
07	Threats, Controls, and Residual Limitations	5
08	Fixed Scenario and Initial State	6
09	Three Outcome Paths	6
10	Proposal Contract	7
11	Identical Digest for Identical Content	8
12	Input Validation in Python	8
13	Policy Gate: No Control Is Optional	9
14	Gate Decision Order and the Revision Problem	9
15	Transactional SQLite Execution — Part One	9
16	Approval Consumption, Audit, and Execution Result — Part Two	10
17	Decision Boundary: Why ALLOW Is Not Execution	12
18	Three Teachable Scenarios and a Real Agent Capability Example	14
19	What Does the Evidence Establish?	15
20	15-Minute Paper Exercise Without Download	15
21	Second 15 Minutes of Practical Exercise	16
22	Answer Key and Scoring Method	16
23	Summary, Sources, and Next Session	16
24	Public Portfolio Edition: License, Ownership, and Provenance	18

Section 01 of 24 — The Boundary Between Proposal, Approval, and Execution

Instructor objective: The learner should be able to state, in one sentence, why an Agent's proposal is not a permit to execute.

Suppose an Agent determines that the number of retries in a service is too low and proposes raising `retry_limit` from 3 to 5. The Agent can describe such a proposal, but it is not allowed to create authority for itself by generating the text "Approved". Approval must come from a person or service that holds authority, through a trusted path. Even after approval, execution must independently re-check the conditions.

Three core terms: PROPOSAL is a description of a change; APPROVAL is conditional permission for that specific change; EXECUTION is a controlled attempt to apply the change. In this session, the Gate output differs from the actual result of execution. **Exercise:** ask the learner: if the Agent says "I approved it myself", which trust boundary is broken?

Scope: The goal of the session is to read and analyze the printed code and to solve the paper exercise; building an executable system is not a requirement of the exercise.

Section 02 of 24 — 90-Minute Teaching Map

Instructor execution table — exactly 90 minutes, with measurable output

Exercises A–F make up exactly 30 minutes of class time; Task G takes place only after class. The code review and the Brownfield workshop are selective, not a complete live implementation.

Time (min)	Sections	Activity	Assessable evidence
0–8	01–03	Identify the authority boundary and record two practical prerequisites	Explanation of one failure mode
8–20	04–06, 08–09	Draw Proposal → Approval → Execution and the trust boundaries	Three-stage diagram with the owner of each decision
20–32	10–12	Analyze the Contract, the canonical digest, and invalid input	List of bound fields and one tamper case
32–45	13–14, 17	Gate evaluation order for Tenant, Scope, Revision, and Replay	Decision table with cause and control priority
45–55	15–16	Atomicity, error, and the difference between ALLOW and Commit	Explanation of rollback and the commit point
55–60	Workshop between 17 and 18	Choose where to inject the Gate into the existing Backend	Three-step migration map without side effects
60–75	20 / A–C	Individual analysis and defense of decisions	Three reasoned answers on the Worksheet
75–90	21 / D–F; 19, 23	Replay/Revision analysis, hand-in, and a two-minute wrap-up	Three-scenario table + two design limitations

Learning matrix: objective → activity → assessment

Design objective	Evidence	Assessment
Design the trust boundary and assign responsibility	Request-path diagram	Task A and explanation of the validity boundary
Design the binding of approval to the exact version of the proposal	Contract + Digest + Revision	Tasks B–D with the reason for each decision
Design a safe execution path and error recovery	Replay and Atomicity	Tasks E–F and two system limitations
Adapt the controls to existing code	Gate injection point and rollout path	Short Brownfield map; supplementary assessment

Instructor note: Begin every concept with the fixed scenario of the controlled change to `retry_limit`; first ask the learner to predict the result, then show the reason for the Gate decision with the example and the code. Sections 7, 18, 22, and 24 are supplementary references; the answer key mentioned in Section 22 is kept as a separate document. In the final 2 minutes, revisit the difference between the Gate decision and the actual execution result.

0–10 minutes: Begin with an unsafe Agent instruction: "Change the value; approval is not needed." Ask the learner why the Agent's text is not an authority reference.

Instructor question: "If the Agent changes the value after approval, what exactly stops execution?" **Expected answer:** binding the valid approval to the proposal's canonical Digest and re-checking it at execution time. Merely saying "the Agent obeys the instructions" is not an acceptable answer.

Section 03 of 24 — Prerequisites and Success Criteria

Measurable outputs: By the end of the session the participant should be able to (1) mark the trust boundary and the Side Effect points of a Coding Agent flow on a diagram; (2) design and justify a Proposal/Approval/Execution contract with Tenant, Scope, Revision, and Digest; (3) reject at least three misuse scenarios — including Replay, a state change after Approval, and Cross-Tenant access — with repeatable tests; and (4) propose the point where the Gate attaches to an existing execution path, together with a Rollback path.

Prerequisite: ability to read basic Python or TypeScript, understanding of a JSON object, and familiarity with the concept of a database transaction. Advanced cryptography knowledge is not required.

Design assessment criterion: In addition to identifying the Proposal, Approval, and Execution stages, the participant must design a verifiable contract for Tenant, Scope, Revision, and Digest; define tests for Replay, Cross-Tenant, and Stale Revision; and document the limitations of the educational version and the Rollback path.

Clearing up a misconception: A Digest is not a digital signature and does not, by itself, prove the approver's identity. A Digest can reveal a change in the content of the proposal only when compared with a value held by a trusted authority. **Question:** if the Agent can change both the proposal and the stored Digest, what guarantee remains?

Section 04 of 24 — Three-Stage Model with a Fixed Example

Proposal: "For tenant-01, change the key `retry_limit` in the logical target `app-config.json` from 3 to 5; the expected Revision is 3." This is merely data from an untrusted source.

Approval: A trusted authority approves proposal `P-001` with a specific Digest, a specific Tenant and target, the authority of the approver, and a valid expiration time. The Agent must not be able to forge the approver identifier or the authoritative record.

Execution: When the request is executed, the system re-checks the same conditions. Only if it is authorized and the Revision matches can the local transaction change the state and consume the approval. **Exercise:** ask: if the

approval was valid yesterday but has expired today, is the proposal still executable? **Answer:** No.

Section 05 of 24 — Two Outcome Families That Must Not Be Mixed

Gate decision: `ALLOW` means the conditions for attempting execution are currently satisfied; `DENY` means the authority or Policy condition is not satisfied; `REVALIDATION_REQUIRED` means the state has become stale and must be evaluated again.

Execution result: `SUCCEEDED` means the Commit or the defined effect has completed; `FAILED` means a definite failure occurred; `UNKNOWN` means it is not known whether the effect actually took place. **Rule:** `ALLOW` ≠ `SUCCEEDED`.

Instructor scenario: The Gate permits execution, but the external service disconnects after the request is sent. Without sufficient evidence, the result may be `UNKNOWN`. Blindly repeating the request is dangerous and requires reconciliation and idempotency mechanisms. The SQLite example of the original document is not a complete state machine for external effects.

Section 06 of 24 — Architecture and Trust Boundary

Educational path:

01	**Agent / LLM** Proposed action — untrusted
02	**Schema + canonical digest** Structure and integrity checks
03	**Trusted approval store** Authority originates outside the Agent
04	**Policy / Tenant / Approver / Expiry gate** Re-check of the authorization context
05	**Revision check** Comparison against the current state
06	**Atomic local transaction** State update + approval consumption + Audit
07	**Execution result** Reported separately from the Gate decision

The Agent only builds the proposal. Schema and Digest control the shape and integrity of the request; authority comes from the approval store and the trusted Policy. Before the state is changed, the current Revision must be checked. In the SQLite example, the state change, the consumption of the approval, and the Audit record are performed in a single local transaction.

Trust assumption: The integrity of the database and the protection of the approval records are environmental prerequisites; the model's text is not a substitute for these controls. **Claim boundary:** The atomicity of a SQLite transaction does not mean that file operations or external-service operations are atomic.

Section 07 of 24 — Threats, Controls, and Residual Limitations

Payload tampering: Digest comparison; conditional on protection of the trusted Digest. **Tenant confusion:** lookup restricted to the Tenant; conditional on a storage-isolation Policy. **Unauthorized approver:** check of the authority reference; real identity verification is outside this example.

Replay: atomic consumption of the approval; the original document shows only a local SQLite example. **Stale Revision:** Compare-and-Swap control; external state may change independently. **Crash between changes:** Commit/Rollback transaction; a Crash-recovery test is not presented in the source.

Source reference: In this section the original document quotes an excerpt of `core/human_authority.py` from AEGIS Agent, including `ApprovalLevel` with the values `AUTO`, `SINGLE_APPROVAL`, `DUAL_APPROVAL`, and `DENY`. **Distinction:** a risk classification alone is not an approval record bound to a Digest.

SOURCE EXCERPT — PINNED REVISION: core/human_authority.py · 4f310113c49c

```
class ApprovalLevel(Enum):
    AUTO = auto()
    SINGLE_APPROVAL = auto()
    DUAL_APPROVAL = auto()
    DENY = auto()
```

This excerpt shows the classification of approval levels, not automatic approval by an Agent. `AUTO` is only the decision of the trusted Policy; `DUAL_APPROVAL` is outside the single-approver example of this session.

Section 08 of 24 — Fixed Scenario and Initial State

HYPOTHETICAL SCENARIO: logical target `app-config.json`, Tenant `tenant-01`, operation `set_config`, key `retry_limit`, current value 3, proposed value 5, and expected Revision 3.

First the Agent builds proposal `P-001`; then a valid approver approves exactly that proposal. When execution is requested, the system re-checks the Tenant, authority, Digest, expiration, and Revision. A role permission (Permission) is not the same as a Digest-bound approval.

Important limitation: The educational SQLite state only simulates the configuration Revision; the code does not open, lock, rewrite, or fsync the real `app-config.json` file. Therefore, a claim of atomic file replacement is incorrect.

Section 09 of 24 — Three Outcome Paths

A — VALID: value 5, matching Digest, valid approval, and current Revision 3. The Gate decision is `ALLOW`; only a successful Commit of the local transaction can lead to `SUCCEEDED`.

B — TAMPERED: after approval, value 5 is changed to 9. The Digest no longer matches; the result is `DENY` with no state change.

C — STALE: the proposal is the same, but the current Revision has become 4. The result is `REVALIDATION_REQUIRED` with no state change. A new proposal based on the fresh Revision, a new Digest, and a new valid approval are required; the old approval must not be used.

Source reference: The original document excerpts `check_permission` from `core/authority_matrix.py`. A role check inside a Tenant is not, by itself, a Digest-bound permit to execute a specific proposal. **Instructor exercise:** for each path, ask first for the Gate result, then for the possible execution result and the reason, separately.

SOURCE EXCERPT — PINNED REVISION: core/authority_matrix.py · f845ab702fba

```
def check_permission(self, tenant_id: str, actor: str, permission: str) -> bool:
    with self._lock:
        roles = self._assignments.get(tenant_id, {}).get(actor, [])
        for r in roles:
            role = self._roles.get(r)
            if role and permission in role.permissions:
                EvidenceLedgerSingleton.append_entry(
                    tenant_id=tenant_id,
                    actor="authority_matrix",
                    action="check_permission",
                    payload={"actor": actor, "permission": permission, "result": True},
                )
                return True
        EvidenceLedgerSingleton.append_entry(
            tenant_id=tenant_id,
            actor="authority_matrix",
            action="check_permission",
            payload={"actor": actor, "permission": permission, "result": False},
        )
        return False
```

This function only checks the role permission within a Tenant; by itself it is not a substitute for an approval bound to a Digest.

Section 10 of 24 — Proposal Contract

Teaching goal: The learner should be able to distinguish between "precisely specifying the request" and "being authorized to execute the request".

An Agent can build a change request but still has no permission to execute it. Before any decision, it must be known who/which request proposed which operation, on which target, with which value, in which Tenant, and based on which Revision. The contract below is a limited educational example; it is not a complete authorization policy.

EDUCATIONAL ADAPTATION — Data Contract; defining a data structure by itself is not authorization to execute:

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Proposal:
    proposal_id: str
    tenant_id: str
    operation: str
    target: str
    key: str
    value: int
    revision: int
```

Explanation for the instructor: `proposal_id` is the proposal identifier; `tenant_id` is the tenant boundary; `operation` is the type of operation; `target` is the logical destination; `key` is the property to be changed; `value` is the requested value; `revision` is the version on which the proposal was built. `frozen=True` restricts ordinary reassignment of the attributes, but it is not proof of identity, validity of the data, or permission to execute.

Hypothetical scenario: `P-001` for `tenant-01` requests that `retry_limit` be changed from 3 to 5, with Revision equal to 3. **Instructor question:** "If only the value is changed to 9, is the previous approval still valid?" **Answer:** No; the proposal has changed, and the Digest match and the approval conditions must be checked again.

Contract boundary: In this session only printable ASCII strings and integers within the JavaScript safe range are accepted; this method is not a general-purpose Canonical JSON standard.

Section 11 of 24 — Identical Digest for Identical Content

Teaching goal: The learner should understand why the approval must be bound to an exact, deterministic representation of the proposal.

If two versions of a proposal differ only in the order of the JSON keys, they should not receive different Digests without reason. For that reason the keys are sorted and the separators are fixed; then SHA-256 is applied to the output bytes. This Digest must be kept together with the approval record in a trusted store; the Agent must not be able to change the reference Digest.

EDUCATIONAL ADAPTATION — source-based educational rewrite; the source's test report alone does not mean independent testing of this rewrite:

```
SAFE_MAX = 9007199254740991

def digest(p: Proposal) -> str:
    validate_contract(p)
    record = {'schema_version': 1, **asdict(p)}
    payload = json.dumps(record, sort_keys=True,
                         separators=(',', ':'),
                         ensure_ascii=True).encode('ascii')
    return hashlib.sha256(payload).hexdigest()
```

Line-by-line explanation: First `validate_contract` checks the type and range of the fields; then `schema_version` is added to the record. `sort_keys=True` fixes the key order, `separators` removes unnecessary whitespace, and `ensure_ascii=True` is consistent with the limited ASCII contract. Finally, SHA-256 produces a hexadecimal Digest.

Dependencies of the example: `import json`, `import hashlib`, and `from dataclasses import dataclass, asdict`; the complete definition of `Proposal` is in Section 10, and that of `validate_contract` is in Section 12. An equal hash alone does not prove the identity or authority of the approver.

Short exercise: Change `value` from 5 to 9 and explain why the approval bound to the previous Digest can no longer be used for the new proposal. This is an educational prediction, not a report of running this example.

Section 12 of 24 — Input Validation in Python

Teaching goal: The learner should explain the difference between validating the shape of the data and the Policy Gate.

Before the Digest is built, data that may be ambiguous or outside the contract must be rejected. In Python, the value `True` belongs to the `int` family; for that reason, using `type(value) is int` — instead of simply accepting via `isinstance` — matters in this contract.

EDUCATIONAL ADAPTATION — source-based educational rewrite; the source's test results alone do not mean independent testing of this rewrite:

```
def validate_contract(p: Proposal) -> None:
    if type(p) is not Proposal:
        raise ValueError('Invalid proposal type')
    fields = asdict(p)
    expected = {'proposal_id', 'tenant_id', 'operation',
                'target', 'key', 'value', 'revision'}
    if set(fields) != expected:
        raise ValueError('Invalid field set')
    for key, value in fields.items():
        if key in ('value', 'revision'):
            if type(value) is not int or not -SAFE_MAX <= value <= SAFE_MAX:
                raise ValueError('Invalid integer')
        elif type(value) is not str or not value or any(
                not 32 <= ord(ch) <= 126 for ch in value):
            raise ValueError('Invalid string')
```

Explanation for the instructor: First the object itself must be exactly of type `Proposal`. Then the set of fields is compared with the contract. `value` and `revision` must be genuine integers within the permitted range; strings must be non-empty and consist only of printable ASCII characters. In the next stage, the Policy Gate applies further semantic restrictions: `value` between 1 and 10 and `revision` non-negative.

Three assessment questions: Is `value=True` accepted? No. Is a Unicode string outside printable ASCII accepted? No. Does passing this Validator mean the request is authorized? No; approval, authority, Tenant, Digest, expiration, Revision, and Policy must still be checked.

Security limitation: This Validator is specific to one fixed dataclass and is not a complete network-request Parser. The related repository samples are only for explaining access control and are not an implementation of this educational Gate.

Section 13 of 24 — Policy Gate: No Control Is Optional

Single reference for these four sections: the standalone educational code `session01_reference.py`; this code is not a direct quotation from GitHub. The example changes only the local SQLite state and does not rewrite the real `app-config.json` file.

Gate order:

1. Contract and type of the Proposal
2. Allowlist of operation, key, and target
3. Valid trusted time
4. Approval record restricted to the Tenant
5. Proposal identifier
6. Digest
7. `APPROVED` status
8. Expiration and target
9. Authority of the approver
10. Revision. A Revision mismatch produces the output `REVALIDATION_REQUIRED`.

Important: The canonical algorithm of this course is custom and makes no claim of conformance with RFC 8785. A Digest is not a signature; the reference Digest must be independent of the Agent.

Section 14 of 24 — Gate Decision Order and the Revision Problem

Three Gate decisions: `DENY` for lack of authority or a mismatch of the approval; `REVALIDATION_REQUIRED` for a stale Revision; `ALLOW` only for permission to attempt local execution. Consumption of the approval (`CONSUMED`) prevents Replay. For each branch, state the Gate result separately from the execution result.

SQL allowlist: A SQL column name cannot be parameterized with a placeholder; therefore only the specific triple (`set_config`, `retry_limit`, `app-config.json`) is allowed. The logical target of the file is mapped to the column `config_state.retry_limit` in this example.

Section 15 of 24 — Transactional SQLite Execution — Part One

Connection setup: `sqlite3.connect(':memory:', isolation_level=None, timeout=5)`, then `BEGIN IMMEDIATE`. In this example SQLite has a single concurrent writer; `timeout` limits the wait for the lock. The condition `WHERE revision=?` also teaches Compare-and-Swap.

Audit: The proposal identifier, Digest, Tenant, value before/after, Revision before/after, time, and result are recorded. A rejected attempt that is rolled back together with the transaction has no durable record; recording it independently requires a separate design.

Section 16 of 24 — Approval Consumption, Audit, and Execution Result — Part Two

Resolving the error-path contradiction: Errors before Commit cause a `rollback` and are then re-raised (`raise`); the error is not swallowed. A Commit error is not necessarily a definite failure or `UNKNOWN`; in an ambiguous situation the state must be examined with Reconciliation. `FAILED` and `UNKNOWN` are defined in the Enum, but this example reports only a successful local Commit as `SUCCEEDED`.

COURSE CODE — TESTED: complete, standalone reference; the 11 `unittest` tests in `tests/test_session01_reference.py` pass (11/11); no Crash Recovery test or real-file test.

```
"""Course-authored reference, NOT a verbatim excerpt from any GitHub repository.
Demonstrates SQLite-local authorization and state transition, not real file I/O.
"""
from dataclasses import dataclass
from enum import Enum
import hashlib, json, sqlite3

SAFE_MAX = 2**53 - 1

class GateDecision(str, Enum):
    ALLOW='ALLOW'; DENY='DENY'; REVALIDATION_REQUIRED='REVALIDATION_REQUIRED'

class ExecutionResult(str, Enum):
    SUCCEEDED='SUCCEEDED'; FAILED='FAILED'; UNKNOWN='UNKNOWN'

@dataclass(frozen=True)
class Proposal:
    proposal_id:str; tenant_id:str; target:str; operation:str; key:str; value:int; revision:int

def validate(p):
    if not isinstance(p,Proposal): raise ValueError('invalid proposal')
    for x in (p.proposal_id,p.tenant_id,p.target,p.operation,p.key):
        if not isinstance(x, str) or not x:
            raise ValueError('invalid ASCII field')
        if any(ord(c)<33 or ord(c)>126 for c in x):
            raise ValueError('invalid ASCII field')
    if type(p.value) is not int or not 1<=p.value<=10: raise ValueError('invalid value')
    if type(p.revision) is not int or not 0<=p.revision<=SAFE_MAX: raise ValueError('invalid revision')

def digest(p):
    validate(p)
    fields = ('proposal_id', 'tenant_id', 'target',
              'operation', 'key', 'value', 'revision')
    payload = {k: getattr(p, k) for k in fields}
    encoded = json.dumps(
        payload, sort_keys=True, separators=(',', ':'),
        ensure_ascii=True
    ).encode('ascii')
    return hashlib.sha256(encoded).hexdigest()

def connect():
    c=sqlite3.connect(':memory:',isolation_level=None,timeout=5)
    c.executescript('''
    CREATE TABLE config_state(tenant_id TEXT PRIMARY KEY,
        retry_limit INTEGER NOT NULL,
        revision INTEGER NOT NULL);
    CREATE TABLE approvals(approval_id TEXT PRIMARY KEY,
        tenant_id TEXT NOT NULL,
        proposal_id TEXT NOT NULL,
        digest TEXT NOT NULL,
        status TEXT NOT NULL,
        expires_at INTEGER NOT NULL,
        approver TEXT NOT NULL,
        target TEXT NOT NULL);
    CREATE TABLE approver_permissions(tenant_id TEXT NOT NULL,
        approver TEXT NOT NULL,
        operation TEXT NOT NULL,PRIMARY KEY(tenant_id,approver,operation));
    CREATE TABLE audit(id INTEGER PRIMARY KEY,
        approval_id TEXT NOT NULL,
        tenant_id TEXT NOT NULL,
        proposal_id TEXT NOT NULL,
```

```
        digest TEXT NOT NULL,
        old_value INTEGER,
        new_value INTEGER,
        revision_before INTEGER,
        revision_after INTEGER,
        at INTEGER NOT NULL,
        result TEXT NOT NULL);
    ''')
    return c

def seed(c,p,now=1000,expiry=2000,approver='instructor',approval_id='A-001'):
    c.execute('INSERT INTO config_state VALUES (?,?,?)',(p.tenant_id,3,3))
    c.execute(
        'INSERT INTO approvals VALUES (?,?,?,?,?,?,?,?)',
        (approval_id, p.tenant_id, p.proposal_id,
         digest(p), 'APPROVED', expiry, approver, p.target)
    )
    c.execute(
        'INSERT INTO approver_permissions VALUES (?,?,?)',
        (p.tenant_id, approver, p.operation)
    )

def gate(c,p,approval_id,trusted_now):
    try: validate(p)
    except (ValueError,TypeError): return GateDecision.DENY
    if type(trusted_now) is not int or trusted_now<0: return GateDecision.DENY
    allowed = ('set_config', 'retry_limit', 'app-config.json')
    if (p.operation, p.key, p.target) != allowed:
        return GateDecision.DENY
    a = c.execute(
        'SELECT proposal_id,digest,status,expires_at,'
        'approver,target FROM approvals '
        'WHERE approval_id=? AND tenant_id=?',
        (approval_id, p.tenant_id)
    ).fetchone()
    if not a: return GateDecision.DENY
    pid,dig,status,expiry,approver,target=a
    if (pid != p.proposal_id or dig != digest(p)
            or status != 'APPROVED'
            or expiry < trusted_now or target != p.target):
        return GateDecision.DENY
    permission = c.execute(
        'SELECT 1 FROM approver_permissions '
        'WHERE tenant_id=? AND approver=? AND operation=?',
        (p.tenant_id, approver, p.operation)
    ).fetchone()
    if not permission: return GateDecision.DENY
    state = c.execute(
        'SELECT revision FROM config_state WHERE tenant_id=?',
        (p.tenant_id,)
    ).fetchone()
    if not state: return GateDecision.DENY
    if state[0] == p.revision:
        return GateDecision.ALLOW
    return GateDecision.REVALIDATION_REQUIRED

def execute(c,p,approval_id='A-001',trusted_now=1000):
    """Errors before commit: rollback and raise. Ambiguous commit errors: raise, reconcile externally.
    Do not assert UNKNOWN on any arbitrary sqlite3 exception: ambiguity must be evidenced.
    """
    c.execute('BEGIN IMMEDIATE')
    try:
        decision=gate(c,p,approval_id,trusted_now)
        if decision!=GateDecision.ALLOW:
            c.rollback(); return decision,None
        old = c.execute(
            'SELECT retry_limit,revision FROM config_state '
            'WHERE tenant_id=?', (p.tenant_id,)
        ).fetchone()
        u = c.execute(
            'UPDATE config_state SET retry_limit=?, '
            'revision=revision+1 '
            'WHERE tenant_id=? AND revision=?',
            (p.value, p.tenant_id, p.revision)
        )
        if u.rowcount!=1:
```

```
            c.rollback(); return GateDecision.REVALIDATION_REQUIRED,None
        u = c.execute(
            "UPDATE approvals SET status='CONSUMED' "
            "WHERE approval_id=? AND tenant_id=? "
            "AND status='APPROVED'",
            (approval_id, p.tenant_id)
        )
        if u.rowcount!=1:
            c.rollback(); return GateDecision.DENY,None
        c.execute(
            'INSERT INTO audit('
            'approval_id,tenant_id,proposal_id,digest,old_value,'
            'new_value,revision_before,revision_after,at,result) '
            'VALUES (?,?,?,?,?,?,?,?,?,?)',
            (approval_id,p.tenant_id,p.proposal_id,digest(p),
             old[0],p.value,old[1],old[1]+1,trusted_now,'SUCCEEDED')
        )
        c.commit()
        return GateDecision.ALLOW,ExecutionResult.SUCCEEDED
    except Exception:
        if c.in_transaction: c.rollback()
        raise

DEFAULT=Proposal('P-001','tenant-01','app-config.json','set_config','retry_limit',5,3)
```

Boundary of the Replay guarantee in this example: The uniqueness of `approval_id` and the change of that approval's status to `CONSUMED` prevent the same identifier from being used again; but in this example there is no uniqueness constraint on the pair (`tenant_id`, `proposal_id`). Therefore, preventing the repeated execution of one proposal with different approvals cannot be concluded from these 11 tests. In a real design, the approval-issuance policy, a suitable unique key, and a duplicate-approval test must be specified.

Boundary of transaction and trust: `BEGIN IMMEDIATE` coordinates the consumption of the approval, the state update, and the Audit record only inside the same SQLite database; the file `app-config.json` does not change in this example. The approver's identity, the permissions, and the clock `trusted_now` are assumed to be trusted application inputs; identity verification, independence of two approvers, security of Policy changes, and tamper-resistant Audit are not proven in this code.

Limits of the evidence and the advanced scenario: The eleven existing tests mainly measure sequential execution, not the race of two Executors, two Approvals for one Proposal, the exact boundary of expiration, or an unknown state after Commit. In an ambiguous Commit error, this example does not return `UNKNOWN`; it raises the error and requires external Reconciliation. Ask the learner to design a negative test and an acceptance criterion for these four cases; do not present them as "tested".

Test evidence: Valid, Tampered, Stale, Replay, Tenant mismatch, Expired, Unauthorized approver, Rollback on Audit error, Bool rejection, Allowlist, and Digest change: 11/11 passed. The tests are in `tests/test_session01_reference.py` next to `session01_reference.py`.

Section 17 of 24 — Decision Boundary: Why ALLOW Is Not Execution

Goal: With a single fixed request, the learner should be able to explain the order of the Gate checks and distinguish the three outputs `DENY`, `REVALIDATION_REQUIRED`, and `ALLOW`.

In the hypothetical example, the Agent registers proposal `P-001` for `tenant-01`: value `retry_limit` from 3 to 5 and expected Revision 3. This proposal is not a permit. The function below describes only the decision logic and performs no real operation on a file or database.

PSEUDOCODE — NOT EXECUTABLE — NOT AN EXECUTOR — COURSE-AUTHORED:

```
function evaluate(proposal, approval, trusted_authority, trusted_now, current_revision):
    if exact_schema_invalid(proposal): return DENY
    if policy_invalid(proposal) or time_invalid(trusted_now): return DENY
    if approval_missing_or_not_approved(approval): return DENY
    if digest_mismatch(proposal, approval): return DENY
    if tenant_or_scope_mismatch(proposal, approval): return DENY
    if expired(approval, trusted_now) or not trusted_authority: return DENY
    if current_revision != proposal.revision: return REVALIDATION_REQUIRED
    return ALLOW
```

Line-by-line explanation: First the shape of the data and the Policy are checked. Then the existence and status of the approval, the Digest binding, the scope of Tenant and operation, the expiration, and the approver's authority are assessed. Only after that is the current Revision compared with the proposal's Revision. If all controls succeed, `ALLOW` is only permission to attempt execution; the success of the change is not yet proven.

Instructor question: "If the proposed value is still 5 but the current Revision has become 4, why is the output not `DENY`?" **Educational answer:** assuming the earlier controls pass, the state change leads to `REVALIDATION_REQUIRED`.

Source limitation: This pseudocode is not a TypeScript implementation; the Runtime, the Validator, the exact Digest serialization, the connection to a Trusted Store, the atomic TypeScript transaction, the consumption of the approval, and the Audit record are not shown in it. Equivalence of execution between Python and TypeScript has not been proven.

Architecture Workshop — Attaching the Policy Gate to a Brownfield System

Problem: An existing Backend service keeps the `retry_limit` setting in the table `service_config`. A Specialized Coding Agent proposes, in order to reduce errors, that the value be changed from 3 to 5. The Agent is allowed only to read code and produce a Proposal; it has no direct access to the Production database, the Migration Runner, or the approval signing key.

Trust boundary: The Agent's output is untrusted data even if it contains the phrase "approved". Only an Approval issued from an independent channel that holds authority, after matching the approver's identity, Tenant, Scope, Digest, expiration, and Revision, is valid. The change must also be executed in an independent transaction together with an Audit record.

Layer in the existing system	Testable responsibility	Failure control
Specialized Coding Agent	Produces a versioned Proposal with a limited diff	No write access or execution of sensitive tools
Proposal API	Canonicalizes and computes the Digest from valid data	Rejects unknown input and an incompatible Schema
Approval Service	Records the decision of a person/service authorized independently of the Agent	Rejects self-approval and an invalid Scope
Policy Gate	Checks Tenant, Digest, Revision, Expiration, Replay, and Allowlist	Fail closed; the ALLOW decision is not yet execution
Transactional Executor	One-time consumption of the Approval, compare-and-swap, and Audit within a transaction	Rollback on error; no partial change applied

Educational integration contract (PSEUDOCODE — NOT EXECUTABLE): This snippet is an integration design, not a replacement for the executable file `session01_reference.py` or its real tests.

```
proposal = proposal_api.validate_and_digest(agent_output)
approval = approval_store.load(proposal.approval_id)  # independent channel
with db.transaction(immediate=True):
    gate.assert_authorized(proposal, approval, current_revision)
    approval_store.consume_once(approval.id)
    config_repo.compare_and_swap(
        tenant=proposal.tenant_id,
        key="retry_limit",
        expected_revision=proposal.expected_revision,
        new_value=5,
    )
    audit_repo.append_execution(proposal.digest, approval.id)
# COMMIT only if every operation succeeds; exceptions must propagate
```

Design test scenarios: (1) different Tenant → `DENY`; (2) value change after Approval → Digest mismatch; (3) stale Revision → Conflict; (4) repeated consumption → Replay denied; (5) Audit write error → Rollback of all changes; (6) the text "approved" in the Agent's response without an independent Approval → `DENY`.

Measurable output: one five-layer Sequence diagram, one short ADR on where the Gate is executed, one Threat/Control Matrix with six scenarios, and a migration plan without breaking the existing API (Feature Flag, regression test, Shadow Evaluation, and Rollback). **Acceptance criterion:** no direct execution path from the Agent to the Executor, and coverage of all six negative tests.

Limitation: The example above is an educational architecture; correctness in a Production environment requires review of real transactions, concurrency, idempotency, access levels, and deployment.

Section 18 of 24 — Three Teachable Scenarios and a Real Agent Capability Example

CONCEPTUAL CLASSROOM DEMONSTRATIONS — NOT TEST EXECUTION RESULTS

CASE 01 / Valid approval: `P-001` for `tenant-01`, value 5, Revision 3, has a matching Digest, a valid approval, and active authority. The expected Gate result is `ALLOW`. Only after the local transaction succeeds can `SUCCEEDED` be reported.

CASE 02 / Tampered proposal: the approval remains for value 5, but the submitted value becomes 9. The Digest no longer matches; the expected result is `DENY` and the state does not change.

CASE 03 / Replay: a single-use approval is presented again after it has been used. The expected result of the second attempt is `DENY`, and the Revision increases only once.

VERIFIED SOURCE EXCERPT — matched to the file and Blob at a specific Commit. AlinaMatrix Enterprise — packages/domain/src/agents.ts

```
export const AGENT_CAPABILITIES: Readonly<Record<AgentName, ReadonlySet<AgentCapability>>> = {
  AlinaArchitect: new Set(["READ_SOURCE", "READ_EVIDENCE", "WRITE_DRAFT"]),
  AlinaGuard:     new Set(["READ_EVIDENCE", "RUN_VALIDATION"]),
  AlinaDocEngine: new Set(["COMPILE_ARTIFACT"]),
  AlinaOptimizer: new Set(["READ_USAGE", "WRITE_CACHE_METADATA"]),
};
```

Source commit URL as printed in the original PDF:

`https://github.com/aminbita162-glitch/AlinaMatrix-Enterprise/blob/47eb2c2a108f91bd2c8d3f0311a79d30cec26971/packages/domain/src/agents.ts`

Blob SHA verified against GitHub:

`ca3ac3ce6672a0590a4198264aac0a298fbd3cb5`

Instructor explanation: `READ_SOURCE` and `WRITE_DRAFT` are Agent capabilities, not human authority to approve a Release or to execute a change. The types `AgentName` and `AgentCapability` are defined in another part of the code.

Section 19 of 24 — What Does the Evidence Establish?

Goal: The learner should distinguish between observation, a permitted claim, and an unproven claim.

Claim	Assessment and scope
The limited classroom example shows the binding of approval to the proposal and the Policy and Revision controls	A limited educational claim consistent with the text of the source; conditional on the same assumptions
Consumption of the approval was demonstrated within a local SQLite transaction	Only within the scope of local SQLite, not all external side effects
The system is cryptographically identity-authenticated	Not proven; do not claim it
A real file was rewritten atomically	Not proven; do not claim it
Python and TypeScript have identical runtime behavior	Not proven; do not claim it
Crash Recovery was tested and verified	Not proven; do not claim it

Independent evidence required: Identity Assurance, management of authorities, multi-process concurrency, transaction durability, Audit retention, distributed effects, and resolution of the `UNKNOWN` state each require their own evaluation and specific evidence.

Instructor question: "If a SQLite test succeeds, can we say that a file change in the operating system is also atomic?" **Answer:** No; these are two different test scopes.

Instructor decision table — fixed scenarios

This table is for the 90-minute review; the execution outcome is `SUCCEEDED` only in the valid scenario and only if the transaction executes successfully.

Scenario	Gate	State change	Execution result	Reason
Valid	ALLOW	Yes, on successful execution	SUCCEEDED	Digest, Policy, and Revision are valid
Tampered: 9	DENY	No	Not executed	Digest mismatch; 9 is within the Policy range
Stale: revision 4	REVALIDATION_REQUIRED	No	Not executed	Revision on which the approval is based is stale
Replay	DENY	No	Not executed	The approval is already CONSUMED
Tenant mismatch	DENY	No	Not executed	Tenant scope does not match
Expired	DENY	No	Not executed	The trusted validity period has passed
Unauthorized approver	DENY	No	Not executed	The authority reference does not allow it

Section 20 of 24 — 15-Minute Paper Exercise Without Download

CLASSROOM FIXTURE — HYPOTHETICAL / PAPER-BASED — NO DOWNLOAD REQUIRED

Initial state: The Tenant is `tenant-01`, the logical target is `app-config.json`, the value is `retry_limit=3`, and the Revision is 3. Proposal `P-001` requests `set_config` for the key `retry_limit` with value 5 and expected Revision 3. Approval `A-001` exists in the trusted store: the Proposal and Tenant identifiers match; the Operation and Target match; the stored Digest is for the original proposal with value 5; the status is `APPROVED`; the approval has not

expired; the approver's authority is active; and the approval is single-use.

Task A — 5 minutes: Identify the three stages Proposal, Approval, and Execution. Underline the fields that must be bound to the approval.

Task B — 5 minutes: For the original proposal, choose one of `ALLOW`, `DENY`, or `REVALIDATION_REQUIRED` and explain why `ALLOW` is still not `SUCCEEDED`.

Task C — 5 minutes: Change only the submitted Proposal value from 5 to 9 and keep the stored Approval unchanged. State the Gate decision and the only control that rejects this change.

Instructor guide: The answers and scoring criteria are not part of this edition; keep them off the learner worksheet.

Section 21 of 24 — Second 15 Minutes of Practical Exercise

Learning goal: The learner should separate the Gate decision from the actual execution result and be able to explain Replay and Revision drift.

Task D — 5 minutes | HYPOTHETICAL: Restore the original `P-001` with value 5 and expected Revision 3, but set the current Revision to 4. First guess the Gate result; then explain why the old approval cannot be used automatically after the Revision changes. Before showing the answer, the instructor should hear each learner's reasoning.

Task E — 5 minutes | HYPOTHETICAL: Assume Revision 3, and imagine that the first authorized local transaction has committed successfully, set `retry_limit` to 5, advanced the Revision to 4, and marked `A-001` as `CONSUMED`. The same approval is submitted again. Analyze the result and whether a second state change is possible. **Teaching point:** the consumed status is checked before the Revision mismatch.

Task F — 5 minutes: Build a three-row table "Original / Tampered / Replay". For each row write the Gate decision, whether the state changes, and the reason. Then explain why the consumption of the approval and the protected state change must be in a single local transaction, and name two limitations of the example.

Hand-in: an annotated table and a short explanation of Atomicity. To carry out the paper exercise, no code download, connection to a real system, or creation of an external Side Effect is needed. Coding extensions are possible only with a complete Fixture approved by the instructor.

Task G — optional instructor extension, 5 minutes, outside the mandatory 30-minute exercise: After the transaction reaches `commit()`, the connection is cut before the acknowledgement is received. What is the safe result to report? Why is automatic Retry without Reconciliation and Idempotency unsafe?

Section 22 of 24 — Answer Key and Scoring Method

The reference answers, the scoring logic, and the scenario key are not included in this edition; they belong in a separate answer-key document. This guide does not contain the exercise answers. The learner version must be distributed separately and without answers.

Section 23 of 24 — Summary, Sources, and Next Session

Provenance register of three code excerpts — GitHub check on 14 October 2026

Source	AEGIS — core/human_authority.py
Commit	`4f310113c49c0ea17c865a5af2670c0a01707655`
Blob SHA	`8e3c6a4defe0b4263ccc8cba1ef87435eee26877`
License / status	Author's own repository (aminbita162-glitch). Proprietary, all rights reserved (repository LICENSE file); excerpt reproduced here by the rights holder.

Source	AEGIS — core/authority_matrix.py
Commit	`f845ab702fbafe1b6b6ea2abca842de50c7bb206`
Blob SHA	`9409ecd6046e6c3d03026c9f01c05fe57a05e7fc`
License / status	Author's own repository (aminbita162-glitch). Proprietary, all rights reserved (repository LICENSE file); excerpt reproduced here by the rights holder.

Source	AlinaMatrix — packages/domain/src/agents.ts
Commit	`47eb2c2a108f91bd2c8d3f0311a79d30cec26971`
Blob SHA	`ca3ac3ce6672a0590a4198264aac0a298fbd3cb5`
License / status	Author's own repository (aminbita162-glitch). Proprietary, all rights reserved (license stated in the repository README); excerpt reproduced here by the rights holder.

Blob confirmation establishes that the excerpt matches the pinned file; it does not certify the security of the whole project.

Five key principles: (1) A Proposal is not an authority reference. (2) An Approval must be bound to the exact data and the Scope. (3) Authority is re-checked at the moment of Execution. (4) The Gate decision is separate from the Execution result. (5) A single-use Approval is consumed together with the protected state change.

Educational architecture reference: AEGIS AGENT and AlinaMatrix Enterprise. All three files on GitHub were re-read at the specified Commit and their Blob identifiers were matched. The educational code and the pseudocode are separate from direct quotations from the repositories.

Official references:

`https://docs.python.org/3/library/sqlite3.html`
`https://docs.python.org/3/library/json.html`
`https://docs.python.org/3/library/hashlib.html`
`https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/isSafeInteger`

Next session: Session 02 — Proposal Contract and Structured Output.

Instructor's final check before the end of the session: Ask each learner to explain one decision path with its reason, not merely repeat the name of the status. In the answer, the connection of the Approval to the Digest, the Revision, and the Scope must be clear; the learner must also explain that `ALLOW` is only permission to enter the execution stage and does not prove the success of the actual execution.

For the Replay scenario, ask the learner to state the difference between "consuming one Approval" and "preventing the repeated execution of a Proposal". If the answer lacks the boundary of the SQLite transaction and of external side effects, make the conceptual correction right there.

Section 24 of 24 — Public Portfolio Edition: License, Ownership, and Provenance

PUBLIC PORTFOLIO EDITION — This English edition is a public portfolio sample prepared by the author for demonstration of educational and architectural work. It preserves the technical and pedagogical content of the original teaching session (the separate answer key referred to in Section 22 is not part of this edition) while removing all personal identifiers of former participants.

Copyright & Ownership

Copyright © 2026 Amin Azimi. All rights reserved.

Author: Amin Azimi — AI Architect — End-to-End System Development. Brand: Azimi Innovation Lab.

Permitted Use (Public Portfolio Context)

- Viewing, reading, and referencing this document on the author's public GitHub portfolio is explicitly permitted.
- Quoting short excerpts with clear attribution to the author and this document is permitted for non-commercial discussion, review, or educational commentary.
- The educational code examples and architectural concepts may be studied and referenced.

Restrictions

- Redistribution of the complete PDF (or substantially complete digital copies) as a standalone product, commercial resale, or republication under another name or brand is not permitted without prior written permission from the copyright holder.
- Removal of author attribution or this provenance section is not permitted.
- Code examples remain educational adaptations only. They carry no warranty and make no claim of fitness for production use, security guarantees, or commercial deployment.

Provenance Statement

Only the three boxes explicitly marked SOURCE EXCERPT (one of them additionally marked VERIFIED) reproduce short excerpts from the author's own GitHub repositories (AEGIS and AlinaMatrix Enterprise), pinned to specific commits and blob SHAs. All other code, pseudocode, diagrams, and explanatory text in this document are original course-authored educational adaptations. Third-party standards and library references remain subject to their respective licenses.

Privacy Note

All personal names of former course participants have been removed from this edition. The document contains no private or identifying information of any individual other than the author.

Contact

Permission requests beyond the scope stated above may be directed to the copyright holder.